Name : Jithin Jyothi

Roll : AM.SC.U4CSE23042

Prompts used :

1. Finding the repo based on the task

Prompt :

I am doing a university case study titled “Design Pattern Identification and Evolution in a Spring Boot-Based E-Commerce Application.” Find an open-source e-commerce application developed using Spring Boot. It should have accessible source code and features such as product management, cart, orders, payments, inventory, customers, notifications, and authentication. Compare suitable repositories against these requirements and recommend the best one. Support the recommendation with repository links and source evidence.

2. Checking if the part 2 changes to make already exist in the project.

Prompt :

Inspect the supplied Shopizer source before I modify it. The repository is https://github.com/shopizer-ecommerce/shopizer.git, using tag/version label 3.2.7. Check whether these requirements are already implemented:

1. Add Razorpay as a payment method.

2. Require category-specific product attributes, such as a warranty for Electronics.

3. Send SMS notifications when an order's status changes.

3. Making the part 1 report.

Prompt :

I have attached my Part 1 report draft, the required template, and the shopizer zip. Correct the report so that the application description, architecture, design patterns, class names, source paths, tables, and evidence match the actual project code. Follow the provided template and preserve correct content. Do not invent missing details or state a pattern not present. Give the correct document.

4. Making the part 2 report.

Prompt ：

Part 1 is complete, now help me with part 2 using this template, findings, and Shopizer zip. The three changes are Razorpay payment integration, mandatory Electronics product-attribute validation, and SMS notification for order-status changes. For each, explain the requirement, existing design, proposed change, affected components, and impact on relevant patterns. Classify each pattern as preserved, modified, extended, replaced, or removed, with a reason supported by code evidence. Continue page numbering after page 8 and do not repeat part 1.

5. Choosing the two patterns for part 3.
   Part 3 – Cross-Language Analysis
   Select 2 significant patterns discovered in Parts 1–2.
   Implement the same problem using:
   - Java
   - Python
   - JavaScript
   - C++

Proceeded with the recommended pattern.

6. Starting the implementation of Observer and Strategy using Python.

   Prompt ：

   Lets start with the Python implementation in VSCode. First define the expected behavior and test cases for both patterns so that we can implement the same functionality in JavaScript, Java and C++ later.

7. After confirming the cases of Python, started with JavaScript implementation.

   Prompt :

   Now, implement the same payment strategy and order status observer problems in JavaScript. Follow the requirements and Python test cases so that the

behaviour remains consistent across languages. Include tests for the same cases and provide the commands needed to run them.

8. After JS implementation, moved to Java implementation.

   Prompt :

   After making the UML diagram, implement the same payment strategy and order status observer problems in Java. Include the test files and exact compilation and execution commands.

9. After JS implementation, moved to C++ implementation.

   Prompt :

   Now, implement the same payment strategy and order status observer problems in C++. Also provide the commands to run it.

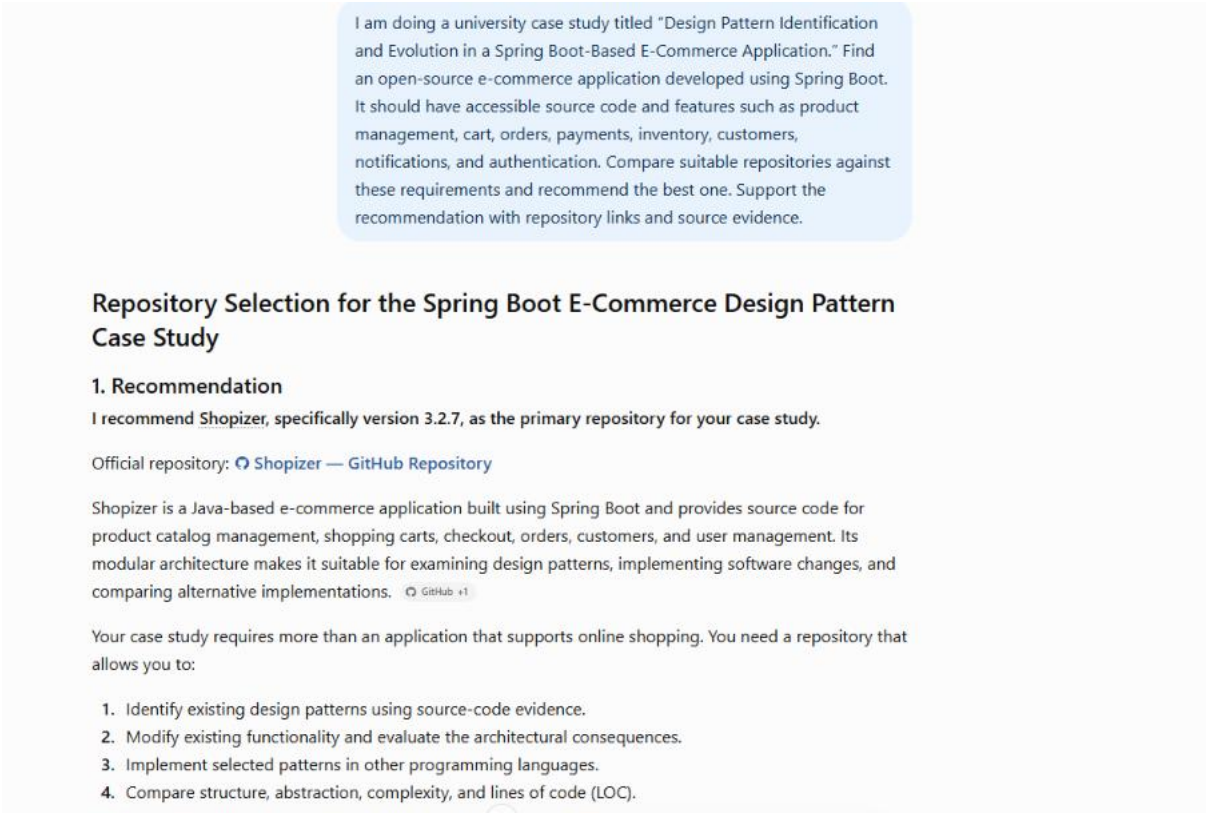
10. Verifying all four language implementation.

    Prompt :

    Inspect the final implementation zip and audit both strategy payment module selection and observer order status SMS notification across Java, Python, JavaScript and C++. Check all the documents and verify the correctness.

LLM outputs :

1.



2. Checking if the part 2 changes to make already exist in the project.

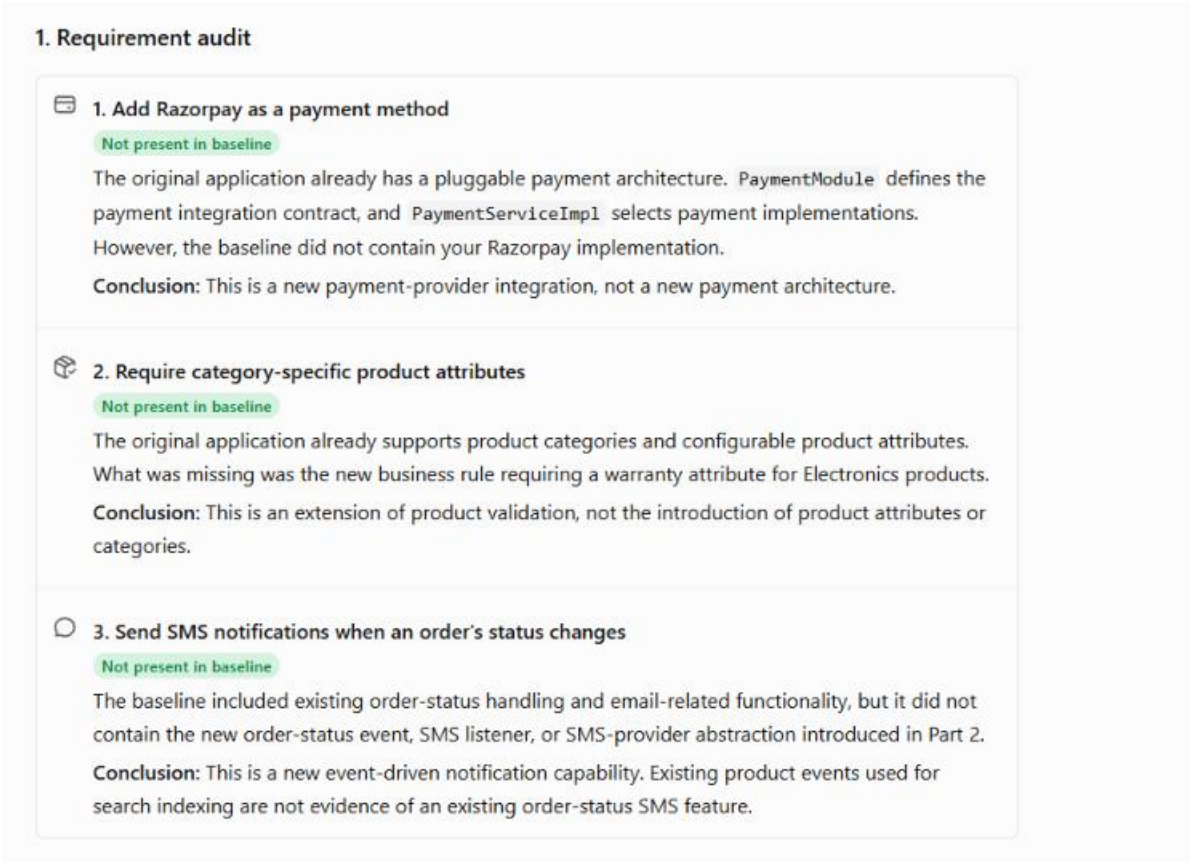
Prompt ：

Inspect the supplied Shopizer source before I modify it. The repository is https://github.com/shopizer-ecommerce/shopizer.git, using tag/version label 3.2.7. Check whether these requirements are already implemented:

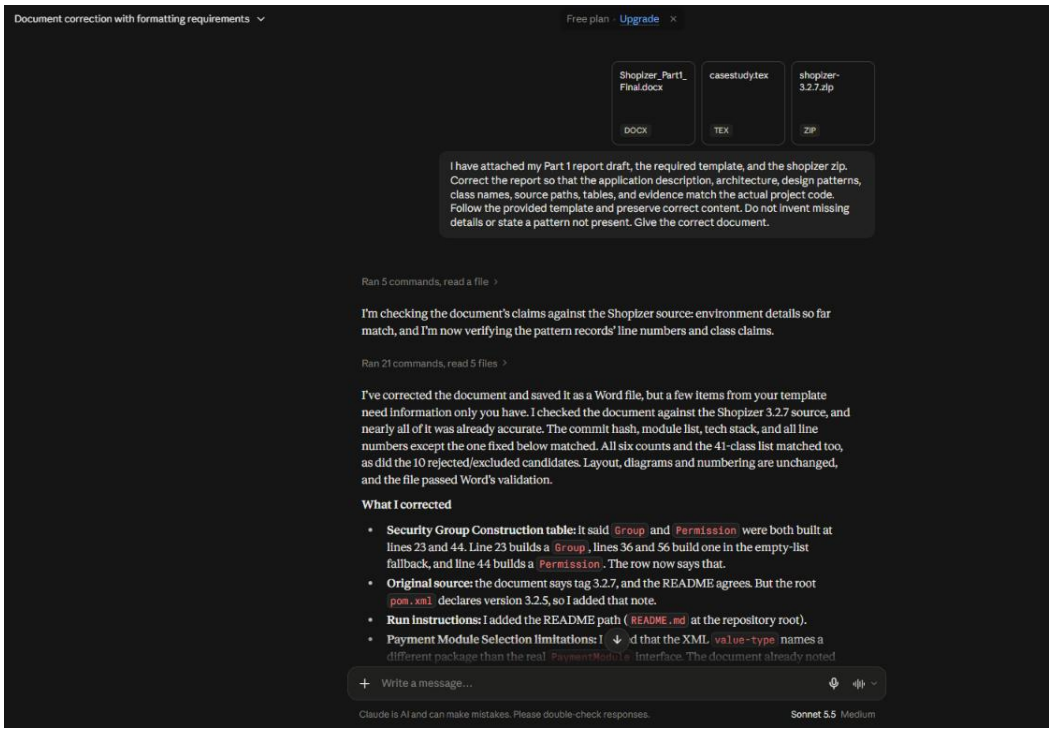
1. Add Razorpay as a payment method.

2. Require category-specific product attributes, such as a warranty for Electronics.

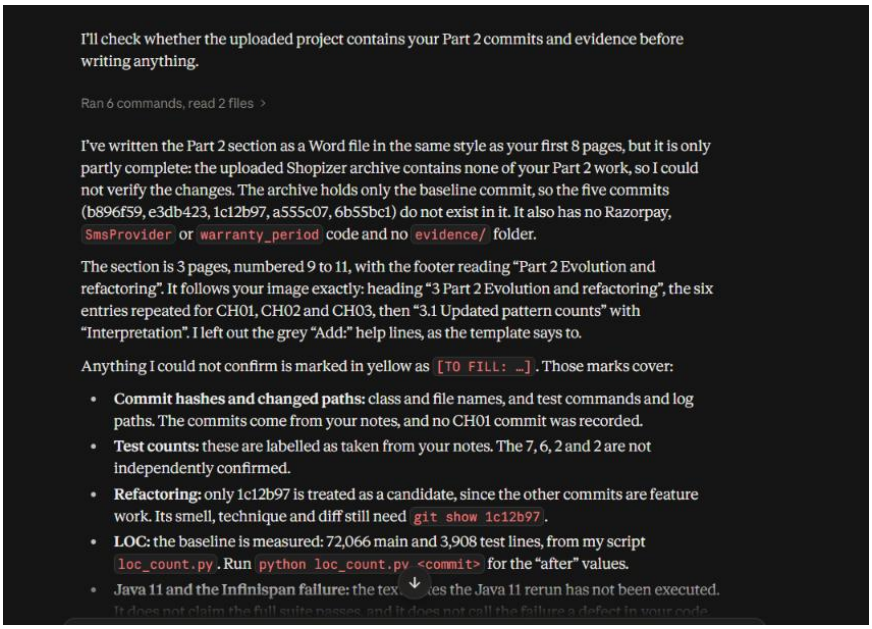
3. Send SMS notifications when an order's status changes.



3. I have attached my Part 1 report draft, the required template, and the shopizer zip. Correct the report so that the application description, architecture, design patterns, class names, source paths, tables, and evidence match the actual project code. Follow the provided template and preserve correct content. Do not invent missing details or state a pattern not present. Give the correct document.



4. Part 1 is complete, now help me with part 2 using this template, findings, and Shopizer zip. The three changes are Razorpay payment integration, mandatory Electronics product-attribute validation, and SMS notification for order-status changes. For each, explain the requirement, existing design, proposed change, affected components, and impact on relevant patterns. Classify each pattern as preserved, modified, extended, replaced, or removed, with a reason supported by code evidence. Continue page numbering after page 8 and do not repeat part 1.



I'll check whether the uploaded project contains your Part 2 commits and evidence before writing anything.

Ran 6 commands, read 2 files >

I've written the Part 2 section as a Word file in the same style as your first 8 pages, but it is only partly complete: the uploaded Shopizer archive contains none of your Part 2 work, so I could not verify the changes. The archive holds only the baseline commit, so the five commits (b896f59, e3db423, 1c12b97, a555c07, 6b55bc1) do not exist in it. It also has no Razorpay, `SmsProvider` or `warranty_period` code and no `evidence/` folder.

The section is 3 pages, numbered 9 to 11, with the footer reading "Part 2 Evolution and refactoring". It follows your image exactly: heading "3 Part 2 Evolution and refactoring", the six entries repeated for CH01, CH02 and CH03, then "3.1 Updated pattern counts" with "Interpretation". I left out the grey "Add:" help lines, as the template says to.

Anything I could not confirm is marked in yellow as `[TO FILL: …]`. Those marks cover:

- **Commit hashes and changed paths:** class and file names, and test commands and log paths. The commits come from your notes, and no CH01 commit was recorded.
- **Test counts:** these are labelled as taken from your notes. The 7, 6, 2 and 2 are not independently confirmed.
- **Refactoring:** only 1c12b97 is treated as a candidate, since the other commits are feature work. Its smell, technique and diff still need `git show 1c12b97`.
- **LOC:** the baseline is measured: 72,066 main and 3,908 test lines, from my script `loc_count.py`. Run `python loc_count.py <commit>` for the "after" values.
- **Java 11 and the Infinispan failure:** the tex…es the Java 11 rerun has not been executed. It does not claim the full suite passes, and it does not call the failure a defect in your code.

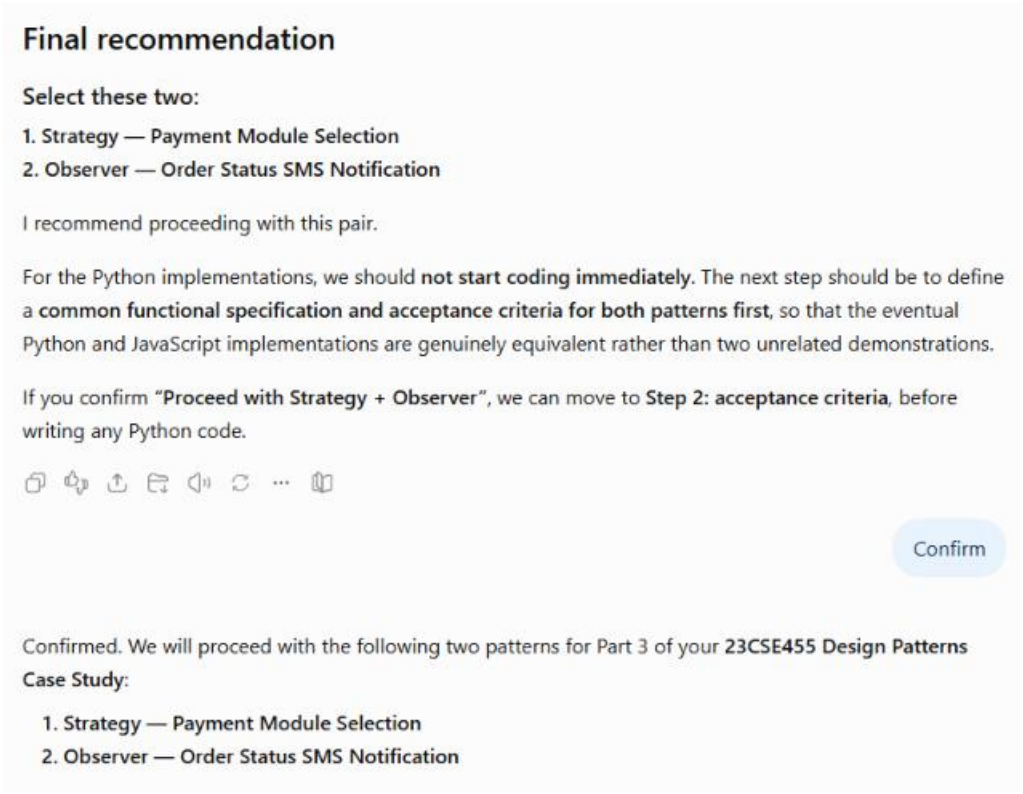
5. Proceeded with recommended pattern after reading through the comparisons provided.

Part 3 – Cross-Language Analysis

Select 2 significant patterns discovered in Parts 1–2.

Implement the same problem using:

- Java
- Python
- JavaScript
- C++



Final recommendation

Select these two:

1. Strategy — Payment Module Selection
2. Observer — Order Status SMS Notification

I recommend proceeding with this pair.

For the Python implementations, we should **not start coding immediately**. The next step should be to define a **common functional specification and acceptance criteria for both patterns first**, so that the eventual Python and JavaScript implementations are genuinely equivalent rather than two unrelated demonstrations.

If you confirm **"Proceed with Strategy + Observer"**, we can move to **Step 2: acceptance criteria**, before writing any Python code.

Confirm

Confirmed. We will proceed with the following two patterns for Part 3 of your **23CSE455 Design Patterns Case Study**:

1. **Strategy — Payment Module Selection**
2. **Observer — Order Status SMS Notification**

6. Lets start with the Python implementation in VSCode. First define the expected behavior and test cases for both patterns so that we can implement the same functionality in JavaScript, Java and C++ later.

Sunday 8:01 PM

DP_Csestudy(1).pdf
PDF

Lets start with the Python implementation in VSCode. First define the expected behavior and test cases for both patterns so that we can implement the same functionality in JavaScript, Java and C++ later.

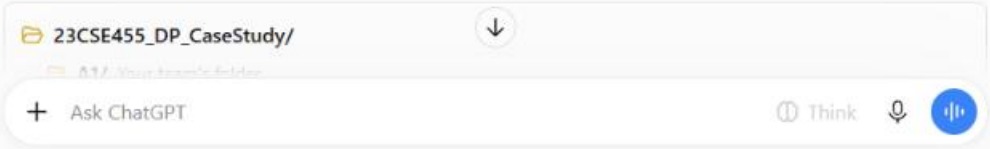
Part 3 — Initial Setup in VS Code

We will start with the Python implementation of **Strategy — Payment Module Selection**, then implement Observer after Strategy has been tested.

I have reviewed your assignment instructions. They specify a particular submission hierarchy, four language implementations (Java, Python, JavaScript and C++), separate production code and tests, and a CSV dataset for cross-language measurements. DP_Csestudy.pdf We should organize the project correctly from the beginning so that you do not need to restructure it before submission.

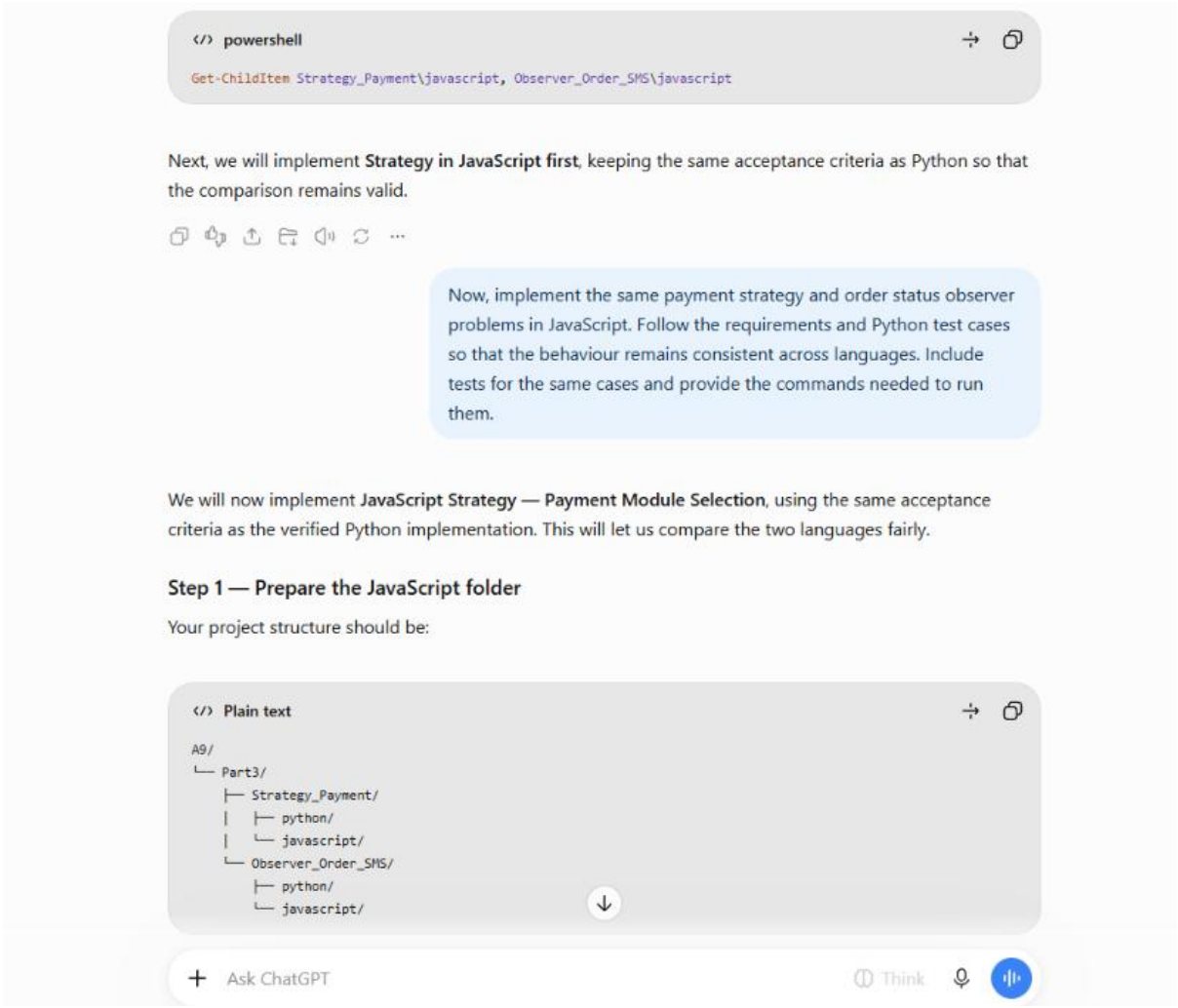
1. Understand where Part 3 belongs

Your assignment specifies the following hierarchy, using `A1` as an example team folder. Replace `A1` with your actual team folder name if different. DP_Csestudy.pdf



7.

Now, implement the same payment strategy and order status observer problems in JavaScript. Follow the requirements and Python test cases so that the behaviour remains consistent across languages. Include tests for the same cases and provide the commands needed to run them.



```
powershell
Get-ChildItem Strategy_Payment\javascript, Observer_Order_SMS\javascript
```

Next, we will implement **Strategy in JavaScript first**, keeping the same acceptance criteria as Python so that the comparison remains valid.

Now, implement the same payment strategy and order status observer problems in JavaScript. Follow the requirements and Python test cases so that the behaviour remains consistent across languages. Include tests for the same cases and provide the commands needed to run them.

We will now implement **JavaScript Strategy — Payment Module Selection**, using the same acceptance criteria as the verified Python implementation. This will let us compare the two languages fairly.

Step 1 — Prepare the JavaScript folder

Your project structure should be:

```
Plain text
A9/
└── Part3/
    ├── Strategy_Payment/
    │   ├── python/
    │   └── javascript/
    └── Observer_Order_SMS/
        ├── python/
        └── javascript/
```

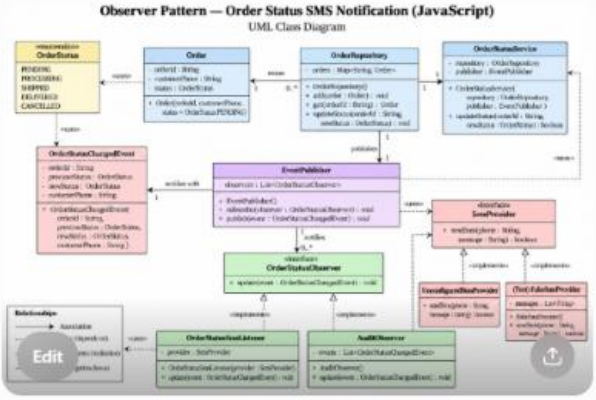
Ask ChatGPT Think

8. After making the UML diagram, implement the same payment strategy and order status observer problems in Java. Include the test files and exact compilation and execution commands.

After pasting, press `Ctrl + S`.

Next task: create the JavaScript Observer UML diagram, then begin the Java implementations. Keep the recorded test results and LOC values tied to the actual terminal evidence you collected.

After making the UML diagram, implement the same payment strategy and order status observer problems in Java. Include the test files and exact compilation and execution commands.



1. JavaScript Observer UML diagram

The diagram above represents the JavaScript Observer implementation, including the publisher, observers, order-status service, repository and SMS-provider abstraction.

For your submission, save the diagram in:

```
+sendSms(phone, message) : bool
```

You can render the editable source using Mermaid Live Editor ↗ and export it as PNG or SVG.

3. Next: Start the Java implementations

We will implement the same two patterns in Java, preserving the functional requirements established for Python and JavaScript.

Step 1 — Create the Java directories

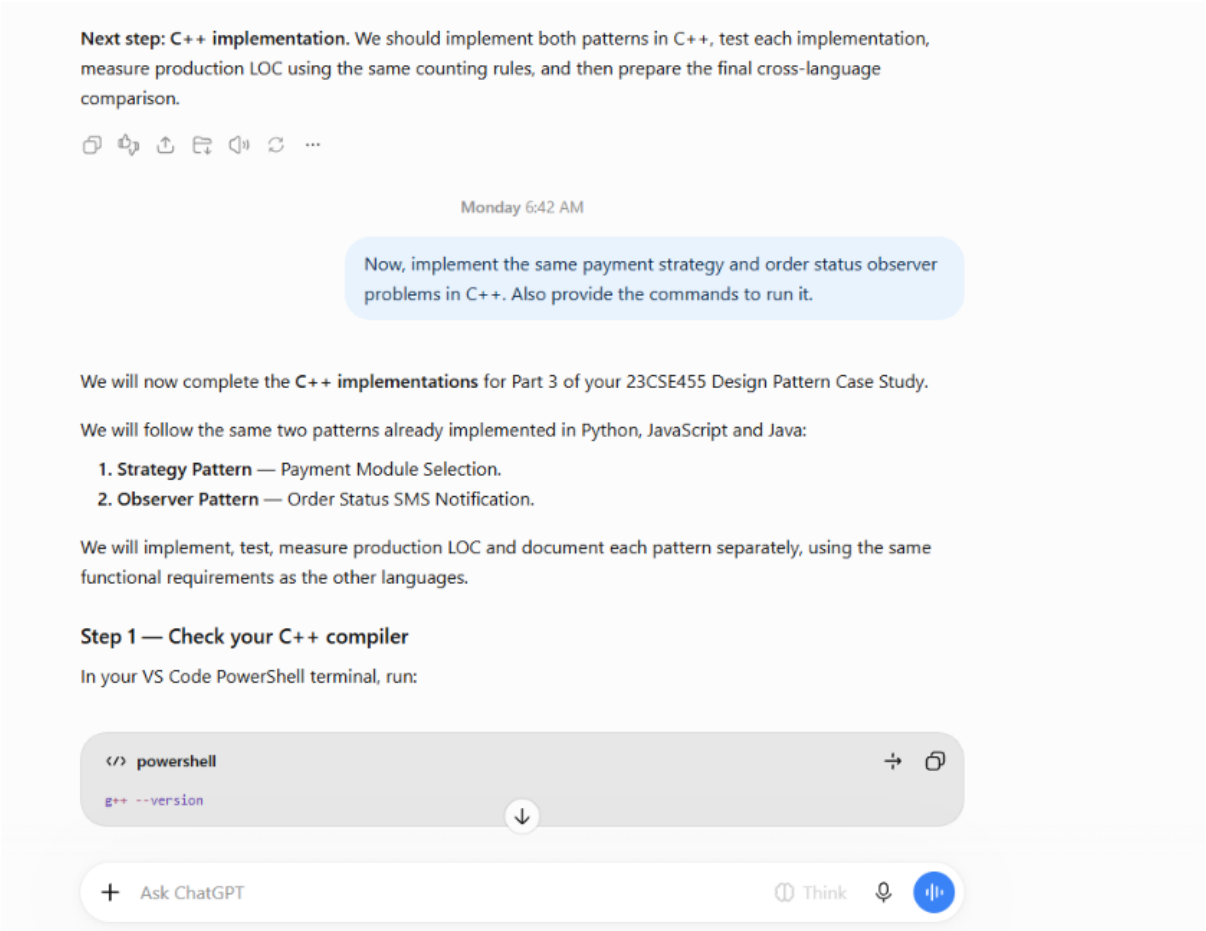
Your current terminal was last in `Observer_Order_SMS\javascript`. In PowerShell, run:

powershell

```
Set-Location ..\..
New-Item -ItemType Directory -Force -Path `
  "Strategy_Payment\java", `
  "Observer_Order_SMS\java"
```

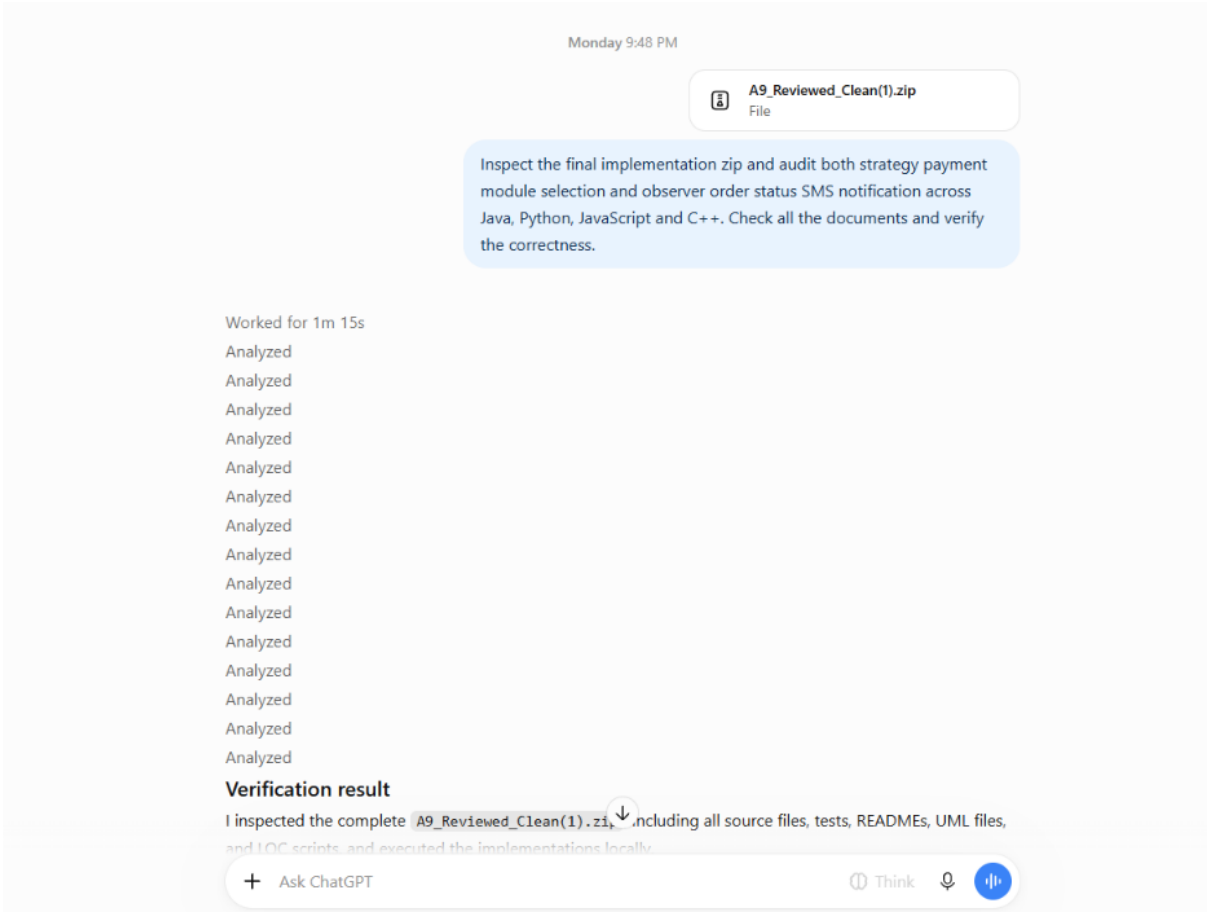
The resulting structure will be:

9. Now, implement the same payment strategy and order status observer problems in C++. Also provide the commands to run it.



10.

Inspect the final implementation zip and audit both strategy payment module selection and observer order status SMS notification across Java, Python, JavaScript and C++. Check all the documents and verify the correctness.



Verification result

I inspected the complete `A9_Reviewed_Clean(1).zip`, including all source files, tests, READMEs, UML files, and LOC scripts, and executed the implementations locally.

Overall verdict

The four-language implementations are functionally correct for both selected patterns.

The two patterns are:

1. **Strategy Pattern — Payment Module Selection**
2. **Observer Pattern — Order Status SMS Notification**

All **8 implementations** compile/run successfully when using the documented/correct commands, and all have passing test suites.

However, the ZIP is **not yet complete as a submission for all 12 expected Part 3 outputs.** The implementation portion is strong, but several required documentation/dataset outputs are absent.

1. Strategy Pattern — Four-language verification

Language	Production LOC	Tests	Result
Java	101	10/10	✅ Pass
Python	80	10/10	✅ Pass
JavaScript	107	10/10	✅ Pass
C++	98	10/10	✅ Pass